BẢN KÊ KHAI SỬ DỤNG AI — TUẦN 5 (WEEK 5 AI USAGE DECLARATION)

Mã Môn học / Lớp: Testing-CQ2023-3-LQVu
Chủ đề Seminar: T07 — CI/CD & Test-Harness Engineering
Hệ thống kiểm thử (SUT): EShop System
Nhóm thực hiện: Nhóm 13
Giai đoạn: Tuần 5

---

1. BẢNG TỔNG HỢP KÊ KHAI SỬ DỤNG AI THEO TỪNG THÀNH VIÊN

> [!IMPORTANT] **Quy định môn học:** Mỗi thành viên đều phải tự khai báo minh bạch công cụ AI mình đã dùng trong tuần. **Nếu thành viên không sử dụng bất kỳ công cụ AI nào trong tuần làm việc, bắt buộc phải ghi rõ: *"Tôi xác nhận không sử dụng AI trong tuần này"*.**

STT	Họ và tên sinh viên	Mã số sinh viên (MSSV)	Trạng thái sử dụng (Có dùng / Không dùng)	Các công cụ AI đã sử dụng	Tỷ lệ đóng góp của AI vào kết quả (%)
1	Nguyễn Ngọc Minh Tuấn	23120102	Có dùng	Claude	40%
2	Dương Nhật Cường	23120223	Có dùng	OpenCode	40%
3	Trần Gia Cường	23120225	Có dùng	Antigravity IDE (Gemini)	35%
4	Nguyễn Gia Huy	23120273	Có dùng	Codex (OpenAI)	40%

---

2. NHẬT KÝ CHI TIẾT QUÁ TRÌNH SỬ DỤNG AI CỦA TỪNG THÀNH VIÊN

2.1. Sinh viên 1: Nguyễn Ngọc Minh Tuấn (MSSV: 23120102)

- **Tình trạng sử dụng:** Có sử dụng
- **Công cụ:** Claude (Anthropic)
- **Mục đích:**
  - Quét nhanh codebase eshop-sut-seminar để hiểu cấu trúc và các điểm lệch so với api_specification.md
  - Hỗ trợ soạn backend/tests/coupon.test.js (Data-Driven Testing với test.each) cho POST /api/apply-coupon
  - Hỗ trợ phân tích lỗi công thức coupon percent ở server.js dòng 398–401 (Failure Mode 2)
- **Câu Prompt tiêu biểu:**

> *"Viết bộ kiểm thử hướng dữ liệu (Data-Driven Testing với test.each) trong file tests/coupon.test.js để kiểm thử toàn diện API POST /api/apply-coupon (các trường hợp coupon hợp lệ, coupon hết hạn, đơn hàng chưa đủ điều kiện tối thiểu)"*

- **Đánh giá kết quả sinh ra:**
  - Quét được toàn bộ codebase và xác định chức năng để chuẩn bị các test cases
  - Hoàn tất việc sinh ra 21 test cases gồm: coupon hợp lệ (7), hết hạn (2), chưa đủ ngưỡng (4), mã không hợp lệ (4 + 1 ca is_active = 0), và nhóm Failure Mode 2 (3 ca kiểm tra bất biến 0 <= discount_amount <= total)
  - Phát hiện thêm lỗi biên C3: server.js dòng 379 dùng > nên đơn đúng bằng min_order_amount bị từ chối
- **Kiểm chứng & Biên tập thủ công:**
  - Tự tay chạy npm test -- tests/coupon.test.js trên máy cá nhân và lưu log vào
  - Đối chiếu kết quả thực tế với bảng dự đoán ở Lượt 2
  - Chụp màn hình terminal và ghi các chỉnh sửa thủ công vào Report

---

2.2. Sinh viên 2: Dương Nhật Cường (MSSV: 23120223)

- **Tình trạng sử dụng:** Có sử dụng
- **Công cụ:** OpenCode
- **Mục đích:**
  - Quét toàn bộ cấu trúc repo EShop (monorepo 4 thành phần) để xác định runtime và cách chạy backend test suite.
  - Thiết kế và sinh file cấu hình CI/CD `eshop-ci.yml` cho GitHub Actions (runner `ubuntu-latest`, Node.js 22).
  - Hướng dẫn tích hợp `actions/cache` cho thư mục `backend/node_modules` và cách băm cache key theo `backend/package-lock.json`.
  - Giải thích lý do không được chạy `npm ci` khi cache HIT (vì `npm ci` sẽ xóa `node_modules`) và cách dùng điều kiện `if: steps.cache.outputs.cache-hit != 'true'`.
  - Review và chẩn đoán lỗi pipeline CI thất bại ở bước `npm ci` (`npm error code EUSAGE`), chỉ ra nguyên nhân gốc là `backend/package-lock.json` bị khuyết entry `@emnapi/core` / `@emnapi/runtime` so với `package.json`.
  - Hướng dẫn khắc phục bằng cách sinh lại lock (`npm install --package-lock-only`) và kiểm chứng lại luồng `npm ci` + `npm test` trên đúng phiên bản Node.js/npm của runner.
- **Câu Prompt tiêu biểu:**

> *"Bạn là 1 chuyên gia DevOps với kinh nghiệm build các CI/CD Pipeline hoàn chỉnh. Task: Quét toàn bộ repo này, khởi tạo thư mục `.github/workflows/` trong repo EShop, xây dựng file cấu hình `eshop-ci.yml` chạy trên runner `ubuntu-latest` với Node.js 22, tích hợp `actions/cache` cho thư mục `backend/node_modules` và ghi nhận hiệu quả tăng tốc (từ ~60s xuống ~5s)."* (sau đó điều chỉnh runtime từ Node.js 20 lên Node.js 22 vì Node.js 20 hiện đã quá cũ và ít được support nữa) *"Pipeline CI bị lỗi như hình, review và report lại giúp tôi."* (kèm ảnh chụp log GitHub Actions thất bại ở bước `Install dependencies (npm ci)`)

- **Đánh giá kết quả sinh ra:**

    - AI quét repo chính xác: nhận diện đúng `backend/package.json` (script test `jest --runInBand --detectOpenHandles --forceExit`), sự tồn tại của `backend/package-lock.json` và 4 test suites, đồng thời chỉ ra repo chưa có thư mục `.github/`.
    - File workflow sinh ra đúng yêu cầu, đặc biệt cảnh báo chính xác cạm bẫy "`npm ci` luôn xóa `node_modules`" — nguyên nhân khiến cache mất tác dụng nếu dùng sai.
    - Với lỗi pipeline: AI chẩn đoán đúng, **không đổ lỗi cho code/test**, mà chỉ ra `package-lock.json` thiếu entry `@emnapi/core` và `@emnapi/runtime` (transitive của Jest 30 qua `@unrs/resolver-binding-wasm32-wasi`, nền tảng `wasm32`), đồng thời truy đúng commit `c7e7d81` làm lock bị lệch.
- **Kiểm chứng & Biên tập thủ công:**
    - Review file chat log `chatlog_cuong_workflow_cache.md` và commit lên GitHub Organization.
    - Tự tái hiện lỗi `npm ci` bằng Node.js `22.17.0` / npm `10.9.2` (khớp runner) để xác nhận thông báo `EUSAGE`.
    - Tự đối chiếu lock file (đếm package, kiểm tra sự tồn tại của `@emnapi/core` và `@emnapi/runtime`) để xác nhận nguyên nhân do AI nêu.
    - Tự chạy lại `npm ci` (exit 0) và `npm test` (4 suites / 6 tests passed) trên lock file đã sinh, rồi tự commit `backend/package-lock.json` và push để pipeline chạy lại.
    - Tự kích hoạt lại workflow CI/CD trên GitHub Actions để kiểm chứng cache HIT và chụp ảnh màn hình minh chứng lưu vào thư mục `screenshots/`.

2.3. Sinh viên 3: Trần Gia Cường (MSSV: 23120225)

- **Tình trạng sử dụng:** Có sử dụng
- **Công cụ:** Antigravity IDE (Gemini)
- **Mục đích:**
    - Tham khảo kiến trúc cấu hình Jest Test Runner và Supertest cho Express backend.
    - Phân tích cơ chế khóa file SQLite (`SQLITE_BUSY: database is locked`) khi chạy kiểm thử song song đa tiến trình (`--maxWorkers=4`).
    - Hướng dẫn giải pháp cấu hình cờ `--runInBand` trong `package.json` để thực thi kiểm thử tuần tự an toàn.
- **Câu Prompt tiêu biểu:**

    > *"Tôi đã chạy npx jest --maxWorkers=4 và nhận được kết quả như trong hình đã chụp. Sao không thấy lỗi: Error: SQLITE_BUSY: database is locked"*

- **Đánh giá kết quả sinh ra:**
    - AI giải thích chính xác nguyên lý phân biệt giữa Shared Lock (khi đọc) và Exclusive Lock (khi ghi) của SQLite, chỉ ra lý do test suite ban đầu không kích hoạt lỗi lock.
    - Hướng dẫn cấu hình cờ `--runInBand --detectOpenHandles --forceExit` chính xác, giúp toàn bộ 4 test suites đều PASS 100%.
- **Kiểm chứng & Biên tập thủ công:**
    - Tự chạy thực nghiệm trong terminal PowerShell để đối chiếu 2 kịch bản: chạy song song (`npx jest --maxWorkers=4` - 3 suites fail vì lỗi lock) và chạy tuần tự (`npm test` - 4 suites pass).
    - Tự tay chụp ảnh màn hình terminal minh chứng lưu vào thư mục `screenshots/` và tự kiểm tra, commit mã nguồn lên GitHub Organization.

2.4. Sinh viên 4: Nguyễn Gia Huy (MSSV: 23120273)

- **Tình trạng sử dụng:** Có sử dụng
- **Công cụ:** Codex (OpenAI)
- **Thời điểm:** 16/10/2026 (giờ Việt Nam)
- **Mục đích:**
  - Tham khảo cách xây dựng công cụ AI Log Triage cho log kiểm thử Jest của EShop.
  - Hỗ trợ viết `.github/scripts/ai_triage.py`: đọc log từ stdin/file, chuẩn bị yêu cầu phân tích nguyên nhân và Cây giả thuyết, xử lý phản hồi API và xuất Markdown.
  - Hỗ trợ viết bộ kiểm tra offline, hướng dẫn chạy trên PowerShell và phân tích lỗi khi thử kết nối API.
  - Điều chỉnh phần kết nối từ OpenAI sang Gemini sau khi lần gọi OpenAI trả lỗi hết credits.
  - Hỗ trợ xây dựng kịch bản Flaky Test do chờ cố định bằng sleep, ca đối chứng dùng await và hướng dẫn tái hiện Failure Mode 3.
- **Câu Prompt tiêu biểu:**

  > "Bạn là một kỹ sư kiểm thử phần mềm có kinh nghiệm xây dựng công cụ AI hỗ trợ phân tích log lỗi trong CI/CD. Hãy đọc nhiệm vụ của tôi trong `Plan/Week5Plan.md` và hỗ trợ viết `.github/scripts/ai_triage.py`: đọc log kiểm thử thất bại từ stdin hoặc file, gọi OpenAI API, xuất Cây giả thuyết gồm hiện tượng, nguyên nhân khả dĩ, bằng chứng, cách kiểm chứng và hướng khắc phục ra terminal hoặc Markdown. Đọc API key từ biến môi trường, bổ sung hướng dẫn chạy và kiểm tra offline. Thực hiện từng nhiệm vụ một, để phần Flaky Test và đẩy nhánh lên GitHub sau."

  - **Prompt bổ sung:** "bắt đầu qua phần flaky test thôi".
- **Đánh giá kết quả sinh ra:**
  - AI hỗ trợ tạo phần triển khai chính của script, bộ kiểm tra và hướng dẫn sử dụng. Script đọc được log Jest thực tế ở chế độ dry run; bộ kiểm tra offline đạt 6/6 test với phản hồi API giả lập.
  - Hướng dẫn ban đầu còn dài; thông báo HTTP 429 ban đầu chưa phân biệt được nguyên nhân. Phần xử lý lỗi được bổ sung mã chi tiết, giúp xác định `credit_balance_exhausted` từ phản hồi OpenAI.
  - Kết quả hiện có xác nhận khả năng đọc log và các luồng xử lý được kiểm tra offline. Chưa có Cây giả thuyết từ LLM thật nên chưa đánh giá được độ chính xác của chẩn đoán hoặc hiệu quả rút ngắn thời gian phân tích.
  - AI tạo demo `backend/demos/flaky-delay.demo.js` và lệnh `test:flaky`. Codex chạy xác nhận: profile fast có 2 ca pass; profile slow có 1 ca fail do sleep và 1 ca pass nhờ await; hai lượt random quan sát được cả pass và fail. Đây là mô phỏng có can thiệp độ trễ, chưa phải số liệu benchmark CI.
- **Kiểm chứng & Biên tập thủ công:**
  - Tự chạy nhóm Jest `Failure Mode 2`, lưu log tại `Weekly_Reports/Week05/coupon_failure_mode2_huy.txt`: 3 test thất bại, 18 test bỏ qua; ghi nhận `discount_amount = -9000000` và `final_amount = 10000000`.
  - Tự chạy script với `--dry-run` trên log đã tạo, lưu ảnh đầu vào và đầu ra để minh chứng việc đọc log.

- Thử gọi OpenAI API trên máy cá nhân và ghi nhận lỗi `credit_balance_exhausted`; bản triển khai hiện tại chuyển sang Gemini để tiếp tục thử nghiệm với quota miễn phí khi có thể.
- Lưu ảnh minh chứng trong `Weekly_Reports/Week05/screenshots/`: `04_ai_triage_offline_tests.png`, `04_ai_triage_input_log.png`, `04_ai_triage_dry_run_02.png`; lưu prompt và quá trình kiểm chứng trong `chat_logs/chatlog_huy_ai_triage_script.md`.
- Tự chạy demo Flaky với profile fast: 2 ca pass; profile slow: ca sleep fail với `Expected: true`, `Received: false`, còn ca await pass. Đã chụp và lưu `05_flaky_test_fast_pass.png`, `05_flaky_test_slow_fail.png` trong thư mục screenshots.
- Phần gọi Gemini thật và đối chiếu chẩn đoán với mã nguồn chưa được kiểm chứng. Kịch bản Flaky tuần 5 đã có mã và minh chứng đối chứng; chạy lặp, đo flake rate trên CI để tuần 6.

---

3. LƯU TRỮ VÀ ĐÍNH KÈM CHAT LOG VỚI AI

Lưu các file xuất lịch sử trò chuyện (Chat Log / Prompt Log) chi tiết với các công cụ AI tại thư mục:

- Thư mục lưu trữ: `Weekly_Reports/Week05/chat_logs/`
  - `chatlog_tuan_prompt_theory.md` (Nguyễn Ngọc Minh Tuấn — Prompt Theory & Data-Driven Testing)
  - `chatlog_cuong_workflow_cache.md` (CI/CD Workflow & Dependency Cache — Dương Nhật Cường; gồm cả phiên điều tra & sửa lỗi pipeline `npm ci` EUSAGE)
  - `chatlog_giacuong_sqlite_lock.md` (Trần Gia Cường — SQLite Lock & Jest Test Runner; gồm cả phiên điều tra & giải thích lỗi `SQLITE_BUSY`)
  - `chatlog_huy_ai_triage_script.md` (Nguyễn Gia Huy — Codex hỗ trợ xây dựng AI Log Triage; kết nối ban đầu OpenAI, hiện chuyển sang Gemini)
  - `chatlog_huy_flaky_test.md` (Nguyễn Gia Huy — Flaky Test do async delay, ca đối chứng await và kết quả chạy của Codex)

---

4. CAM KẾT LIÊM CHÍNH HỌC THUẬT

Chúng tôi cam đoan những thông tin kê khai trên là hoàn toàn trung thực. Toàn bộ mã nguồn, tài liệu và phân tích kỹ thuật do AI hỗ trợ (nếu có) đều đã được các thành viên trong nhóm trực tiếp đọc hiểu, kiểm chứng bằng thực nghiệm và chịu trách nhiệm hoàn toàn về tính chính xác.

Xác nhận của các thành viên trong nhóm:

Họ và tên	Chữ ký số / Xác nhận điện tử	Ngày ký
Nguyễn Ngọc Minh Tuấn	`Đã ký`	`16/10/2026`
Dương Nhật Cường	`Đã ký`	`16/10/2026`
Trần Gia Cường	`Đã ký`	`15/10/2026`
Nguyễn Gia Huy	`Đã ký`	`16/10/2026`
